

Foldseek: The Missing Manual

February 20, 2026

Contents

1	Introduction to Foldseek	6
1.1	Structural Representation and 3Di	6
1.2	Search and Clustering Backbone	6
1.3	Structural Scoring Modes	6
1.4	Multimer Extension	7
1.5	Database and Index Model	7
1.6	Reading Order	7
2	Foldseek User Guide	7
2.1	Platform and System Requirements	7
2.1.1	Quick CPU Capability Checks	7
2.1.2	GPU Requirements	8
2.2	Memory Planning for Large Searches	8
2.3	Installation Paths	8
2.3.1	Precompiled Binaries	8
2.3.2	Conda Installation	9
2.3.3	Build from Source (Linux CPU)	9
2.3.4	Build from Source (Linux GPU)	9
2.3.5	Build from Source (macOS)	9
2.3.6	Build Customization	9
2.4	Operational Checklist After Installation	9
2.5	FAQ and Troubleshooting	10
2.5.1	What does Foldseek hit probability represent?	10
2.5.2	Why can ranking change when --sort-by-structure-bits changes?	10
2.5.3	When should I use --alignment-type 1 (TM-align) instead of 2 (3Di+AA)?	10
2.5.4	Why do I get warnings about C-alpha data or disabled thresholds?	10
2.5.5	How do I apply U and T transforms to coordinates?	10
2.5.6	How can I align only specific known query-target pairs?	10
3	Foldseek Manual	10
3.1	Workflow Selection	11
3.2	Shared Execution Model	11
3.3	High-Impact Global Options	11
3.4	Output and Reporting Path	12
3.5	Advanced Reading	12
3.6	Module Manual	12
3.6.1	Easy Workflows	13
3.6.1.1	easy-search	13
3.6.1.2	easy-cluster	13
3.6.1.3	easy-rbh	14
3.6.1.4	easy-multimersearch	14
3.6.1.5	easy-multimercluster	14
3.6.1.6	When to Drop to Low-Level Commands	14

3.6.2	Structure Search Modules	14
3.6.2.1	Pipeline Model	14
3.6.2.2	search	15
3.6.2.3	structurealign	15
3.6.2.4	structurerescorediagonal	16
3.6.2.5	talign	16
3.6.2.6	lolalign	16
3.6.2.7	rbh	16
3.6.2.8	samplemulambda	16
3.6.2.9	Practical Composition Patterns	16
3.6.3	Structure Clustering Modules	17
3.6.3.1	cluster	17
3.6.3.2	clust	17
3.6.3.3	Clustering Semantics	18
3.6.3.4	Result Interpretation	18
3.6.3.5	Related Modules	18
3.6.4	Multimer Modules	18
3.6.4.1	Complex Search Pipeline	18
3.6.4.2	multimersearch	19
3.6.4.3	expandmultimer	19
3.6.4.4	scoremultimer	19
3.6.4.5	createmultimerreport	21
3.6.4.6	multimercluster	21
3.6.4.7	Easy Multimer Wrappers	22
3.6.5	Structure Manipulation Modules	22
3.6.5.1	createdb	22
3.6.5.2	createsubdb	23
3.6.5.3	compressca	23
3.6.5.4	convertalis	23
3.6.5.5	convert2pdb	23
3.6.5.6	aln2tmscore	24
3.6.5.7	result2profile	24
3.6.5.8	Typical Data-Preparation Flow	24
3.6.6	Database Management Modules	24
3.6.6.1	Canonical Database Layout	24
3.6.6.2	Core File Formats	25
3.6.6.3	Channel Payload Semantics	26
3.6.6.4	C-alpha Storage Modes (X_ca)	26
3.6.6.5	databases	26
3.6.6.6	createindex	27
3.6.6.7	createclusearchdb	27
3.6.6.8	Practical Layout Strategies	28
3.6.6.9	Minimal End-to-End Example	28

4	Foldseek Expert Manual	28
4.1	3Di Encoding Engine	28
4.2	Prefilter and Candidate Generation	28
4.3	Alignment Core: 3Di, 3Di+AA, and TM-align	29
4.4	TM-score and LDDT in Filtering and Ranking	29
4.5	Multimer Assignment and Complex Scoring	29
4.6	Data Layout, Storage Format, and Memory Behavior	30
4.6.1	On-disk Containers and Sidecars	30
4.6.2	C-alpha Encoding Trade-offs	30
4.6.3	Indexed Targets and Exclusion Flags	30
4.6.4	Cluster-search Data Model	30
4.7	GPU Paths and Constraints	30
4.8	Practical Tuning Order	31
5	Foldseek Developer Manual	31
5.1	Command Registration and Ownership	31
5.2	Parameter System	31
5.3	Workflow Layer	32
5.4	Algorithm and Utility Layer	32
5.5	Core Libraries and Shared Engines	33
5.6	End-to-End Data Flow (Developer View)	33
5.7	Editing Guidelines	33
5.8	Debugging Map	34
5.9	Documentation Sync Rule	34

1 Introduction to Foldseek

Foldseek is a structure search and clustering system that keeps the throughput profile of sequence search while preserving the discriminative power needed for structural biology. It does this by converting protein structures into a structural alphabet, then running a cascaded search pipeline that aggressively prunes candidates before expensive scoring. The result is that search, clustering, and multimer analysis share one backbone, but each mode applies different scoring, filtering, and reporting layers.

This manual is organized around that backbone. The practical entry points are described in the manual and module chapters, while algorithmic and implementation details are centralized in the expert manual and developer manual.

1.1 Structural Representation and 3Di

Foldseek does not compare raw coordinates in the prefilter. During `createdb`, each chain is converted into synchronized data streams: amino-acid sequence, 3Di sequence, C-alpha coordinates, and headers. The 3Di stream is produced by feature extraction from local geometric context followed by embedding and discretization in `foldseek/lib/3di/structureto3di.cpp`. This representation is compact enough for fast k-mer indexing and prefiltering, but still captures local structural context that sequence-only scoring misses.

A key design choice is that Foldseek keeps AA and 3Di views aligned at the residue level. This enables `--alignment-type 2` (3Di+AA) to combine structural and sequence evidence in a single alignment stage instead of forcing separate post-hoc rescoring. The same design also allows fast fallback paths (`--alignment-type 0` or `1`) without changing database layout.

1.2 Search and Clustering Backbone

The core search path is staged:

1. K-mer driven candidate generation on structural descriptors.
2. Ungapped and/or diagonal rescoring to cut candidate volume.
3. Full alignment (`structurealign` or `talign`) with optional TM-score/LDDT gating.
4. Optional post-processing such as multimer assignment, report creation, clustering, or conversion.

The workflow layer in `foldseek/src/workflow/StructureSearch.cpp` wires these stages and adjusts behavior based on options like `--prefilter-mode`, `--alignment-type`, and `--gpu`. The same structural backbone is reused by `search`, `cluster`, `rbh`, and `multimer` workflows, which is why tuning one stage often affects multiple commands.

1.3 Structural Scoring Modes

Foldseek exposes three primary alignment modes through `--alignment-type`:

- `0` (3Di alignment): fastest structural-only local alignment path.
- `1` (TM alignment): TM-align-based global structural alignment.
- `2` (3Di+AA): combined local scoring, default for many workflows.

TM-score and LDDT can be used as explicit thresholds (`--tmscore-threshold`, `--lddt-threshold`) and also influence ranking when structure-bit sorting is enabled (`--sort-by-structure-bits 1`). Those interactions are not purely cosmetic; they affect which hits survive and how hits are ordered.

1.4 Multimer Extension

Foldseek-Multimer extends the same single-chain backbone by adding chain expansion, chain-to-chain assignment, and complex-level scoring stages. In practice, `multimersearch` and `multimercluster` call additional modules (`expandmultimer`, `scoremultimer`, internal filtering/reporting) after base alignment so that complex-level quality is computed from concrete chain assignments rather than inferred from monomer hits.

1.5 Database and Index Model

Foldseek databases are split by data role (`DB`, `DB_ss`, `DB_ca`, `DB_h`, plus indices). This split is central to performance tuning: index and coordinate exclusions can reduce memory and I/O, but they also remove capabilities needed by specific ranking or alignment settings. The practical rules and exact file layout are documented in Database Management, especially Core File Formats, and the runtime implications are documented in Expert Manual: Data and Performance.

1.6 Reading Order

For new users, start with Manual, then go to Easy Workflows and Structure Search. For optimization and interpretation, continue with Expert Manual. For source-level debugging or extension work, use Developer Manual.

2 Foldseek User Guide

This chapter focuses on operational readiness: platform prerequisites, installation strategies, memory planning, and troubleshooting patterns that affect real runs. For command selection and workflow design, continue to the manual.

2.1 Platform and System Requirements

Foldseek supports modern Linux and macOS environments. On x86_64 platforms, performance depends strongly on SIMD support. AVX2 is the fastest CPU path, while SSE4.1 and SSE2 compatibility paths exist for older hardware. ARM64 and PPC64LE are also supported with architecture-specific SIMD backends.

For production usage, validate CPU capabilities before benchmarking. A run that silently falls back to a lower SIMD tier can look like a configuration bug when it is actually a hardware-path mismatch.

2.1.1 Quick CPU Capability Checks

Linux:

```
[ $(uname -m) = "x86_64" ] && echo "64bit: Yes" || echo "64bit: No"
grep -q avx2 /proc/cpuinfo && echo "AVX2: Yes" || echo "AVX2: No"
```

```
grep -q sse4_1 /proc/cpuinfo && echo "SSE4.1: Yes" || echo "SSE4.1: No"
grep -q sse2 /proc/cpuinfo && echo "SSE2: Yes" || echo "SSE2: No"
```

macOS:

```
[ $(uname -m) = "x86_64" ] && echo "64bit: Yes" || echo "64bit: No"
sysctl machdep.cpu.leaf7_features | grep -q AVX2 && echo "AVX2: Yes" || echo
"AVX2: No"
sysctl machdep.cpu.features | grep -q SSE4.1 && echo "SSE4.1: Yes" || echo
"SSE4.1: No"
```

2.1.2 GPU Requirements

Foldseek supports CUDA acceleration for search workflows and ProST5-assisted 3Di prediction. Full speed is typically achieved on Ampere-generation or newer NVIDIA GPUs, while older Tesla-generation hardware can still run with reduced throughput.

For GPU search and GPU-backed createdb runs, verify both driver/runtime compatibility and the command path (-gpu, plus workflow-specific constraints such as prefilter mode behavior).

2.2 Memory Planning for Large Searches

Memory footprint is dominated by database representation and index settings. A practical baseline for structure-aware runs is:

$$\text{RAM} \approx (6 \text{ bytes C-alpha} + 1 \text{ byte 3Di} + 1 \text{ byte AA}) \times \text{total residues}$$

For AFDB50-scale workloads, this can exceed 100 GB depending on index and scoring mode. If you disable structure-bit sorting (--sort-by-structure-bits 0) and rely on paths that do not require C-alpha coordinates during ranking, effective runtime memory can drop substantially.

A safe planning approach is:

1. Estimate peak RAM from residue counts.
2. Decide whether coordinate-aware ranking/metrics are required.
3. Choose index subset/exclusion options that remain compatible with downstream scoring and output requirements.

2.3 Installation Paths

2.3.1 Precompiled Binaries

Linux AVX2:

```
wget https://mmseqs.com/foldseek/foldseek-linux-avx2.tar.gz
tar xvzf foldseek-linux-avx2.tar.gz
export PATH=$(pwd)/foldseek/bin/:$PATH
```

Linux SSE4.1:

```
wget https://mmseqs.com/foldseek/foldseek-linux-sse41.tar.gz
tar xvzf foldseek-linux-sse41.tar.gz
export PATH=$(pwd)/foldseek/bin/:$PATH
```

macOS universal build:

```
wget https://mmseqs.com/foldseek/foldseek-osx-universal.tar.gz
tar xvfz foldseek-osx-universal.tar.gz
export PATH=$(pwd)/foldseek/bin/:$PATH
```

2.3.2 Conda Installation

```
conda install -c conda-forge -c bioconda foldseek
```

2.3.3 Build from Source (Linux CPU)

```
git clone https://github.com/steineggerlab/foldseek.git
cd foldseek
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=RELEASE -DCMAKE_INSTALL_PREFIX=. ..
make -j$(nproc)
make install
export PATH=$(pwd)/bin/:$PATH
```

2.3.4 Build from Source (Linux GPU)

```
conda create -n nvcc -c conda-forge cuda-nvcc cuda-cudart-dev libcublas-dev
libcublas-static cuda-version=12.6 cmake
conda activate nvcc
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=RELEASE -DCMAKE_INSTALL_PREFIX=. -DENABLE_CUDA=1 -
DCMAKE_CUDA_ARCHITECTURES="75;80;86;89;90" ..
make -j8
make install
export PATH=$(pwd)/bin/:$PATH
```

2.3.5 Build from Source (macOS)

Clang path:

```
brew install cmake libomp zlib bzip2
./util/build_osx.sh PATH_TO_FOLDSEEK_REPO OUTPUT_DIR
```

GCC path:

```
brew install cmake gcc@11 zlib bzip2
CC="gcc-14" CXX="g++-14" cmake -DCMAKE_BUILD_TYPE=RELEASE -
DCMAKE_INSTALL_PREFIX=. ..
```

2.3.6 Build Customization

Most relevant MMseqs2 CMake flags apply to Foldseek. For example, Google Cloud Storage support for createdb can be enabled using `vcpkg` and `-DHAVE_GCS=1`.

2.4 Operational Checklist After Installation

Before starting large jobs:

1. Confirm foldseek is on PATH and binary architecture matches host capabilities.
2. Run a small createdb and search smoke test.
3. Validate temporary storage path throughput and capacity.
4. If using GPU, verify CUDA visibility and run one GPU-enabled search trial.

2.5 FAQ and Troubleshooting

2.5.1 What does Foldseek hit probability represent?

Hit probability is an empirical estimate of true-positive likelihood as a function of structural bit score, calibrated on benchmark distributions. It is useful for ranking interpretation, not a direct replacement for domain-specific validation.

2.5.2 Why can ranking change when `--sort-by-structure-bits` changes?

With structure-bit sorting enabled, ranking uses structural quality terms derived from TM/LDDT-aware scoring. Disabling it reverts ordering closer to raw bit-score behavior. E-values are not necessarily affected in the same way as ranking order.

2.5.3 When should I use `--alignment-type 1` (TM-align) instead of 2 (3Di+AA)?

Use TM-align mode when global superposition quality is the primary objective. Use 3Di+AA when balancing structural and sequence evidence for high-throughput screening and clustering pipelines.

2.5.4 Why do I get warnings about C-alpha data or disabled thresholds?

Some scoring and filtering paths require coordinate channels (`_ca`). If the database layout omits required coordinate components, Foldseek disables incompatible threshold/ranking features to prevent invalid outputs.

2.5.5 How do I apply U and T transforms to coordinates?

Foldseek reports transform parameters for superposition. For direct coordinate transformation workflows, you can use custom scripts or generate superposed outputs through `convert2pdb/convertalis` modes that already apply transforms.

2.5.6 How can I align only specific known query-target pairs?

Construct a custom prefilter DB containing only the desired pair IDs, then run `structurealign` on that prefilter result. A practical template is:

```
foldseek createdb inputs1/ db1
foldseek createdb inputs2/ db2

# Build a pair list mapped to internal IDs, then:
foldseek tsv2db keys.tsv pref --output-dbtype 7
foldseek structurealign db1 db2 pref aln
foldseek convertalis db1 db2 aln aln.m8
```

This approach avoids full all-vs-all candidate generation when pair identities are known in advance.

3 Foldseek Manual

This chapter is the operational map for Foldseek. It does not duplicate command pages; it explains which command family to choose, how stages connect, and which options govern behavior across modules.

3.1 Workflow Selection

Goal	Primary entry point	Next chapter
Single-chain structure search from raw files	<code>easy-search</code>	Easy Workflows
Single-chain search from existing DBs	<code>search</code>	Structure Search
Reciprocal best-hit screening	<code>easy-rbh</code> or <code>rbh</code>	Easy Workflows / Structure Search
Single-chain structural clustering	<code>easy-cluster</code> or <code>cluster</code>	Easy Workflows / Structure Clustering
Complex-level (multimer) search	<code>easy-multimersearch</code> or <code>multimersearch</code>	Multimer Modules
Complex-level clustering	<code>easy-multimercluster</code> or <code>multimercluster</code>	Multimer Modules
Build/maintain databases and indexes	<code>createdb</code> , <code>createindex</code> , <code>databases</code>	Structure Manipulation, Database Management

The easy workflows are wrappers around lower-level commands. They are best for starting quickly from PDB/mmCIF inputs, while low-level commands are better for controlled pipelines, reproducibility, and performance tuning.

3.2 Shared Execution Model

Most workflows pass through the same stages: database preparation, candidate generation, alignment, and conversion/reporting. This is why options such as `-s`, `--prefilter-mode`, `--alignment-type`, and `--sort-by-structure-bits` are visible across many commands and have similar effects.

In `foldseek/src/workflow/StructureSearch.cpp`, the workflow selects prefilter and alignment subcommands dynamically. In multimer workflows (`foldseek/src/workflow/MultimerSearch.cpp`, `foldseek/src/workflow/MultimerCluster.cpp`), the same base stages are followed by chain expansion and complex-level scoring.

3.3 High-Impact Global Options

Option	Where it matters most	Practical effect
<code>-s</code>	Search and cluster workflows	Higher sensitivity increases candidate volume and runtime.
<code>--prefilter-mode</code>	<code>search</code> , <code>multimersearch</code> , <code>easy workflows</code>	Controls whether the pipeline uses k-mer prefilter,

Option	Where it matters most	Practical effect
		ungapped-only, no prefilter, or mixed mode.
<code>--alignment-type</code>	Search, clustering, multimer	Chooses 3Di-only, TM-align, or 3Di+AA alignment engine.
<code>--sort-by-structure-bits</code>	Search/alignment workflows	Enables ranking that uses structural quality terms in addition to bit score.
<code>--tmscore-threshold,</code> <code>lddt-threshold</code>	<code>-- structurealign,</code> <code>structurerescorediagonal,</code> <code>clustered/multimer flows</code>	Hard filters on structural quality; can change both recall and ranking.
<code>--gpu</code>	Search workflows, ProstT5-backed createdb	Enables CUDA paths where implemented; may impose prefilter constraints.
<code>--index-subset,</code> <code>exclude</code>	<code>--index- createindex</code>	Trade memory and build size against downstream feature availability.

3.4 Output and Reporting Path

Most search and clustering commands produce internal result databases. Text, HTML, SAM, and superposed structure outputs are generated in later conversion steps, typically via `convertalis`, `createmultimerreport`, or `convert2pdb`. For reproducible pipelines, treat conversion as a separate stage instead of embedding output concerns into search parameter tuning.

3.5 Advanced Reading

For algorithm details and performance behavior, continue with the Expert Manual. For source-code ownership, module boundaries, and extension points, continue with the Developer Manual.

3.6 Module Manual

The following module chapters are the canonical command reference for Foldseek. They are organized by workflow role and intentionally share options where they share execution stages.

Module	Focus
Easy Workflows	High-level commands from raw structures to final outputs.
Structure Search	Core search/alignment commands and RBH.
Structure Clustering	Cascaded and graph-based structural clustering.

Module	Focus
Multimer Modules	Complex-level search, scoring, expansion, and clustering.
Structure Manipulation	Database creation, conversion, coordinate and output utilities.
Database Management	Downloaded datasets, indexing, and cluster-search DB preparation.

3.6.1 Easy Workflows

The easy workflows are front doors that accept structure files directly and orchestrate database creation, search/alignment, and output conversion. They trade fine-grained control for speed of setup and are ideal when you want reliable defaults with minimal command composition.

Internally, these workflows are orchestrated in `foldseek/src/workflow/Easy*.cpp` and shell templates in `foldseek/data/`. If you need strict stage control, use the low-level commands in Structure Search, Structure Clustering, and Multimer Modules.

3.6.1.1 easy-search

`easy-search` runs end-to-end single-chain structural search from raw structure input to text output.

Usage

```
foldseek easy-search <i:PDB|mmCIF|.gz> ... <i:PDB|mmCIF|.gz>|<i:stdin>
<i:targetFastaFile|.gz>|<i:targetDB> <o:alignmentFile> <tmpDir> [options]
```

High-impact options are `-s`, `--prefilter-mode`, `--alignment-type`, `--tmscore-threshold`, `--lddt-threshold`, `--format-mode`, `--format-output`, and `--gpu`.

`--alignment-type` controls whether hits are finalized by 3Di local alignment (0), TM-align (1), or combined 3Di+AA (2). `--prefilter-mode` determines how aggressively candidates are pruned before alignment. `--format-mode 3` produces HTML output and `--format-mode 5` writes superposed C-alpha PDB views.

3.6.1.2 easy-cluster

`easy-cluster` performs structure clustering from raw files and produces representative/member outputs.

Usage

```
foldseek easy-cluster <i:PDB|mmCIF|.gz> ... <i:PDB|mmCIF|.gz> <o:clusterPrefix>
<tmpDir> [options]
```

This workflow wraps search, alignment filtering, and `clust`. The most important controls are `-c`, `--cov-mode`, `--cluster-mode`, `--alignment-type`, and quality thresholds (`--tmscore-threshold`, `--lddt-threshold`).

3.6.1.3 easy-rbh

easy-rbh computes reciprocal best hits through a bidirectional structure-search workflow.

Usage

```
foldseek easy-rbh <i:queryFastaFile1[.gz|.bz2]> <i:targetFastaFile[.gz|.bz2]>|  
<i:targetDB> <o:alignmentFile> <tmpDir> [options]
```

This is appropriate when orthology-style reciprocal filtering is required but you still want the structural scoring stack available through --alignment-type, TM-score, and LDDT settings.

3.6.1.4 easy-multimersearch

easy-multimersearch is the multimer counterpart of easy-search: it accepts raw complex files and runs complex-aware expansion, assignment, and reporting.

Usage

```
foldseek easy-multimersearch <i:PDB|mmCIF[.gz]> ... <i:PDB|mmCIF[.gz]>|<i:stdin>  
<i:targetFastaFile[.gz]>|<i:targetDB> <o:outputFileName> <tmpDir> [options]
```

In addition to standard search controls, multimer workflows expose --min-assigned-chains-ratio, --monomer-include-mode, --expand-multimer-evaluate, and --multimer-report-mode.

3.6.1.5 easy-multimercluster

easy-multimercluster performs clustering at complex level.

Usage

```
foldseek easy-multimercluster <i:PDB|mmCIF[.gz]> ... <i:PDB|mmCIF[.gz]>  
<o:clusterPrefix> <tmpDir> [options]
```

Compared with easy-cluster, this workflow adds complex-aware filters such as --multimer-tm-threshold, --chain-tm-threshold, and --interface-lddt-threshold.

3.6.1.6 When to Drop to Low-Level Commands

Use easy workflows when you need rapid execution from raw inputs. Switch to low-level commands when you need to reuse intermediate DBs, enforce exact prefilter/alignment stages, tune memory/index compatibility, or insert custom conversion/scoring steps between stages.

3.6.2 Structure Search Modules

This chapter covers the core single-chain search stack. The key distinction is orchestration versus execution: search is a workflow driver, while structurealign, structurerescorediagonal, and tmalign are lower-level alignment/rescoring modules.

In source terms, orchestration is in foldseek/src/workflow/StructureSearch.cpp, and alignment kernels are in foldseek/src/strucclustutils/ plus foldseek/src/commons/.

3.6.2.1 Pipeline Model

The standard path is prefilter -> optional diagonal rescoring -> alignment. --prefilter-mode governs which prefilter stages run, and --alignment-type chooses the final alignment

kernel. The workflow can change behavior under GPU mode; for example, the code path in `StructureSearch.cpp` forces ungapped prefilter defaults when needed.

`--sort-by-structure-bits` changes ranking from plain bit score to a structure-aware term. Because that ranking depends on TM/LDDT information, disabling structure-bit sorting changes how threshold options interact with final ordering.

3.6.2.2 search

Usage

```
foldseek search <i:queryDB> <i:targetDB> <o:alignmentDB> <tmpDir> [options]
```

`search` is the canonical entry point for DB-to-DB structural search. Important controls:

Option	Effect
<code>-s</code>	Primary sensitivity/speed tradeoff in candidate generation.
<code>--prefilter-mode</code>	0: k-mer+ungapped, 1: ungapped only, 2: nofilter, 3: ungapped+gapped.
<code>--alignment-type</code>	0: 3Di, 1: TM-align, 2: 3Di+AA.
<code>--sort-by-structure-bits</code>	Enables structure-aware ranking.
<code>--tmscore-threshold, --lddt-threshold</code>	Structural quality filtering.
<code>--cluster-search</code>	Representative-first search for clustered targets.
<code>--gpu</code>	CUDA search path when supported by the current build.

`search` should be preferred over direct low-level modules unless you intentionally control intermediate artifacts.

3.6.2.3 structurealign

Usage

```
foldseek structurealign <i:queryDB> <i:targetDB> <i:prefilterDB> <o:resultDB> [options]
```

`structurealign` consumes a candidate list (`prefilterDB`) and performs full structural alignment on those pairs. It is the right tool when you already own candidate generation, or when you need to re-align candidate pairs under different thresholds without rerunning prefilter.

Notable behavior from `structurealign.cpp`:

- TM/LDDT thresholding is enforced during alignment output.
- Some option combinations are normalized with warnings, especially around `--sort-by-structure-bits` versus lightweight alignment-output modes.

3.6.2.4 structurerescorediagonal

Usage

```
foldseek structurerescorediagonal <i:queryDB> <i:targetDB> <i:prefilterDB>  
<o:resultDB> [options]
```

This module rescales candidate diagonals using structural scoring and can apply TM/LDDT gating earlier than full workflow composition. It is useful when you want to refine candidate quality in a modular pipeline before deciding on final alignment/reporting stages.

3.6.2.5 talign

Usage

```
foldseek talign <i:queryDB> <i:targetDB> <i:prefilterDB> <o:resultDB> [options]
```

talign runs TM-align directly on candidate pairs. Use it when global structural superposition quality is the primary objective and you already have candidate pairs prepared.

Important options are `--tmscore-threshold`, `--tmscore-threshold-mode`, `--talign-hit-order`, and `--talign-fast`.

3.6.2.6 lolalign

lolalign is a source-visible alignment module (foldseek/src/FoldseekBase.cpp) for local distance log-odds optimization. In current packaged Foldseek releases, this command may be unavailable even though it appears in source.

When enabled in your build, it follows the low-level aligner input pattern:

```
foldseek lolalign <i:queryDB> <i:targetDB> <i:prefilterDB> <o:resultDB> [options]
```

3.6.2.7 rbh

Usage

```
foldseek rbh <i:queryDB> <i:targetDB> <o:alignmentDB> <tmpDir> [options]
```

rbh runs reciprocal best-hit filtering on top of the structural search stack. It is the low-level counterpart to easy-rbh, with full DB-based workflow control.

3.6.2.8 samplemulambda

Usage

```
foldseek samplemulambda <i:queryDB> <i:targetDB> <o:resultDB> [options]
```

samplemulambda estimates score-distribution parameters from shuffled/randomized samples and is mainly for calibration or diagnostics. The main control is `--n-sample` (random sample count).

3.6.2.9 Practical Composition Patterns

A common advanced pattern is to keep stages explicit:

```
foldseek search queryDB targetDB alnDB tmp  
foldseek convertalis queryDB targetDB alnDB result.tsv --format-output  
query,target,alntmscore,lddt,evalue,bits
```

For explicit candidate reuse:

```
foldseek structurerescorediagonal queryDB targetDB prefilterDB rescoredDB
foldseek structurealign queryDB targetDB rescoredDB alnDB
```

This separation is especially useful for benchmarking alternative scoring thresholds or alignment modes on the same candidate set.

3.6.3 Structure Clustering Modules

Foldseek clustering is built on the same search/alignment backbone used for hit finding, followed by graph or greedy clustering in `clust`. The high-level command (`cluster`) runs this sequence end-to-end, while `clust` performs clustering from an existing result graph.

3.6.3.1 cluster

Usage

```
foldseek cluster <i:sequenceDB> <o:clusterDB> <tmpDir> [options]
```

`cluster` runs candidate generation, alignment filtering, and clustering in one workflow. It is the preferred command when you start from a structure DB and want production clustering output.

Core controls:

Option	Effect
-s	Search sensitivity before clustering.
-c and --cov-mode	Coverage gating before edges become cluster links.
--min-seq-id	Sequence-identity threshold for accepting edges.
--cluster-mode	0: set-cover, 1: connected component, 2/3: greedy by length.
--single-step-clustering	Disables cascaded workflow and runs one clustering pass.
--cluster-steps	Number of cascaded stages in cascaded mode.
--cluster-reassign	Reassignment pass to correct cascade artifacts.

Although the command name says sequence DB, in Foldseek this DB includes structural channels (`_ss`, `_ca`) used by structural scoring modules.

3.6.3.2 clust

Usage

```
foldseek clust <i:sequenceDB> <i:resultDB> <o:clusterDB> [options]
```

clust performs only the clustering stage from a precomputed edge/result DB. Use this when you want to rerun cluster strategy without recomputing search/alignment.

The most relevant options are `--cluster-mode`, `--max-iterations`, `--similarity-type`, and optional weighting knobs (`--weights`, `--cluster-weight-threshold`).

3.6.3.3 Clustering Semantics

Set-cover mode tends to produce compact representative sets. Connected-component mode favors transitive closure and typically creates larger, looser clusters. Greedy length-prioritized modes are useful when representative length should dominate assignment order.

In practical tuning, the edge-generation parameters (`-c`, `--cov-mode`, `--min-seq-id`, and structural thresholds inherited from search) usually affect outcomes more strongly than switching between clustering algorithms on the same edge set.

3.6.3.4 Result Interpretation

Cluster output is stored as a cluster DB. Typical post-processing is:

```
foldseek createsubdb clusterDB sequenceDB repDB
foldseek convert2fasta repDB rep.fasta
```

For adjacency-style exports, convert through standard result/TSV tooling depending on downstream format requirements.

3.6.3.5 Related Modules

- Raw-input wrapper: `easy-cluster`
- Complex-level clustering: `multimercluster`
- Index/layout preparation for repeated search+cluster loops: Database Management

3.6.4 Multimer Modules

Foldseek-Multimer extends the single-chain Foldseek backbone into complex-level comparison. It still starts from chain-to-chain candidates, but adds complex-aware expansion, assignment discovery, global superposition scoring, and interface-aware filtering before final reporting.

Workflow orchestration is implemented in `foldseek/src/workflow/MultimerSearch.cpp`, `foldseek/src/workflow/MultimerCluster.cpp`, and shell templates under `foldseek/data/`. Assignment search, geometric clustering, and all multimer-specific filters are implemented in `foldseek/src/strucclustutils/expandmultimer.cpp`, `scoremultimer.cpp`, and `MultimerUtil.h`. `createmultimerreport.cpp` is a reporting layer that restructures already-scored assignments.

3.6.4.1 Complex Search Pipeline

In the default `multimersearch` workflow (`foldseek/data/multimersearch.sh`), the stages are executed in this order:

1. `search`: run the standard Foldseek chain-level search backbone.
2. `expandmultimer`: convert sparse chain hits into complex-consistent chain-pair candidate lists.
3. `structurealign` or `talign`: realign expanded chain pairs when expansion is enabled.

4. `scoremultimer`: construct chain assignments, score complex-level superposition, apply multimer filters, and emit complex-aware alignment records.
5. Optional conversion/reporting (`convertalis`, `createmultimerreport`) in the easy wrapper.

This staged design postpones one-to-one chain assignment until enough evidence has been collected. A single early chain hit does not immediately lock the complex mapping.

3.6.4.2 `multimersearch`

Usage

```
foldseek multimersearch <i:queryDB> <i:targetDB> <o:alignmentDB> <tmpDir>
[options]
```

`multimersearch` is the main DB-to-DB multimer pipeline. Besides standard structure-search options, the multimer-specific controls that change behavior most are `--expand-multimer-evaluate`, `--min-assigned-chains-ratio`, and `--monomer-include-mode`.

Implementation detail that matters for tuning: in non-exhaustive mode, the workflow performs an initial fast search and then realigns expanded chain pairs before complex scoring. In exhaustive mode (`--exhaustive-search 1`), the expansion/re-alignment branch is skipped and scoring runs directly on exhaustive pairwise alignments.

3.6.4.3 `expandmultimer`

Usage

```
foldseek expandmultimer <i:queryDB> <i:targetDB> <i:alignmentDB> <o:prefilterDB>
[options]
```

`expandmultimer` is the candidate-space completion step before assignment scoring. It reads query and target .lookup mappings to recover complex membership, then executes the following logic:

1. For each query complex, collect target complex IDs that have at least one chain hit in `alignmentDB`.
2. For each collected target complex, generate all query-chain × target-chain pairs for that complex pair.
3. Write per-query-chain candidate target-chain lists into `prefilterDB` for downstream alignment/scoring.

This step deliberately expands sparse chain evidence into complex-consistent candidate sets so later assignment can test complete mappings instead of isolated local hits.

3.6.4.4 `scoremultimer`

Usage

```
foldseek scoremultimer <i:queryDb> <i:targetDb> <i:alignmentDB> <o:complexDB>
[options]
```

`scoremultimer` is the core multimer algorithm. It parses chain-level alignments with backtraces, builds candidate chain-to-chain alignment objects, clusters them into assignment hypotheses, computes complex-level superposition/TM, and applies multimer-specific filters.

3.6.4.4.1 Assignment construction

For each query complex and target complex pair, `scoremultimer` (`ComplexScorer` in `scoremultimer.cpp`) converts chain-level records into `ChainToChainAln` objects. Each object stores matched C-alpha coordinates, chain keys, chain-level TM score, and the 12-parameter rigid transform vector (u rotation and t translation).

Before clustering, transforms are standardized component-wise (z-score style, with low-variance dimensions damped) so geometric distance between alignments is comparable across parameters.

3.6.4.4.2 Assignment clustering and one-to-one consistency

Assignment discovery uses a DBSCAN-like clustering layer (`DBSCANCluster`):

- `--min-assigned-chains-ratio` sets the minimum cluster size as `ceil(query_chain_count * ratio)`.
- A reciprocal-best-hit-style pruning keeps alignments whose chain TM score is within $0.7 * \max(\text{best}(\text{query_chain}), \text{best}(\text{target_chain}))$.
- Distance is computed in transform space. If a cluster contains repeated query or target chains, nearest-neighbor truncation enforces one-to-one chain usage.

Each surviving cluster is converted into one `Assignment`: matched residues from member chain alignments are concatenated, then a complex-level rigid transform and TM score are recomputed over the merged coordinate set.

Complex TM is normalized both ways:

$$qTM = tmscore * \min(qLen, tLen) / qLen, tTM = tmscore * \min(qLen, tLen) / tLen$$

so asymmetric complex sizes remain interpretable.

3.6.4.4.3 Filtering order in `scoremultimer`

Filtering is executed in a strict order (`ComplexFilter::filterAssignment`), and early failures stop evaluation:

1. Minimum aligned chains (`--min-aligned-chains`).
2. Complex-level TM gate (`--multimer-tm-threshold`, interpreted by `--cov-mode`).
3. Complex coverage gate (`-c, --cov-mode`).
4. Chain-level TM gate (`--chain-tm-threshold`), also interpreted by `--cov-mode`.
5. Interface gate (`--interface-lddt-threshold`) using query-defined interface residues and LDDT on matched interface atoms.

Interface residues are precomputed on the query complex from inter-chain C-alpha proximity (8 Å cutoff constant in code). If only one chain is aligned and `--interface-lddt-threshold > 0`, the assignment is rejected.

After filtering, only the best assignment per target complex is retained for output (selected by adjusted coverage according to `--cov-mode`).

3.6.4.4.4 Output fields produced by scoremultimer

In addition to chain-level alignment fields, scoremultimer appends multimer fields consumed by convertalis and createmultimerreport, including:

- complexqtmscore, complexttmscore
- complexu, complext
- qcomplexcoverage, tcomplexcoverage
- qchaintms, tchaintms
- interfaceLddt
- complexassignid

3.6.4.5 createmultimerreport

Usage

```
foldseek createmultimerreport <i:queryDb> <i:targetDb> <i:complexDB>  
<o:complexFile> [options]
```

createmultimerreport groups chain-level records by assignment ID and emits one complex-level row per assignment. It is a formatter, not a rescoring stage.

The report columns are:

Column	Meaning
query_complex, target_complex	Complex identifiers parsed from chain headers.
query_chains, target_chains	Chain-name lists assigned in this solution.
qTM, tTM	Complex TM normalized by query and target lengths.
u, t	3x3 rotation and translation vector of complex superposition.
qComplexCov, tComplexCov	Coverage fractions computed during filtering.
qChainTms, tChainTms	Comma-separated per-chain TM values.
interfaceLddt	Interface LDDT for the selected assignment.
assId	Assignment identifier (stable within each query complex result set).

3.6.4.6 multimercluster

Usage

```
foldseek multimercluster <i:sequenceDB> <o:clusterDB> <tmpDir> [options]
```

multimercluster runs multimersearch followed by clustering (clust) on multimer-filtered edges. The command sets stricter runtime defaults than generic search (setMultimerClusterDefaults in MultimerCluster.cpp):

- `--multimer-tm-threshold 0.65`
- `--chain-tm-threshold 0.001`
- `--interface-lddt-threshold 0.5`

These gates act on different failure modes and are intentionally complementary:

Threshold	What it screens
<code>--multimer-tm-threshold</code>	Global complex-level alignment quality.
<code>--chain-tm-threshold</code>	Per-chain geometric compatibility within assignments.
<code>--interface-lddt-threshold</code>	Interface preservation quality across assigned complexes.

3.6.4.7 Easy Multimer Wrappers

- `easy-multimersearch`: raw structure input to complex search output.
- `easy-multimercluster`: raw structure input to complex clusters.

These wrappers are convenient for direct file-based workflows, but the core algorithmic stages remain `multimersearch/scoremultimer/multimercluster`. For deeper algorithm context, see Multimer Assignment and Complex Scoring.

3.6.5 Structure Manipulation Modules

This chapter covers modules that prepare structure databases, transform structural data layouts, and convert internal result DBs into analysis outputs.

3.6.5.1 `createdb`

Usage

```
foldseek createdb <i:directory|.tsv>|<i:PDB|mmCIF|.gz|tar|.gz|DB> ... <i:PDB|mmCIF|.gz|tar|DB> <o:sequenceDB> [options]
```

`createdb` is the ingestion point for Foldseek. It parses structures, builds synchronized AA/3Di/CA/header channels, and writes the DB layout consumed by all search and clustering modules.

The exact on-disk file specification for these channels is documented in Database Management: Core File Formats and Database Management: C-alpha Storage Modes.

Important options:

Option	Role
<code>--db-extraction-mode</code>	0: chain-level entries, 1: interface extraction mode.
<code>--input-format</code>	Force parser mode when autodetection is not sufficient.
<code>--file-include, --file-exclude</code>	Regex-based filtering for directory/tar inputs.

Option	Role
<code>--prostt5-model</code>	Enables 3Di prediction from sequence input using ProstT5.
<code>--gpu</code>	Enables GPU path for ProstT5 inference when available.
<code>--coord-store-mode</code>	C-alpha storage encoding (float or uint16 diff).
<code>--write-lookup, --write-mapping</code>	Metadata mapping outputs for downstream integration.

3.6.5.2 createsubdb

Usage

`foldseek createsubdb <i:subsetFile|DB> <i:DB> <o:DB> [options]`

`createsubdb` creates a keyed subset from an existing Foldseek DB while preserving synchronized structure channels. Typical uses are extracting representatives after clustering or constructing curated subsets for downstream searches.

High-impact options are `--subdb-mode` (copy vs soft-link index view) and `--id-mode` (select by internal keys or FASTA identifiers from `.lookup`).

3.6.5.3 compressca

Usage

`foldseek compressca <i:DB> <o:caDB> [options]`

`compressca` rewrites C-alpha coordinate storage, typically to reduce I/O and footprint. `--coord-store-mode` controls encoding (1 float, 2 diff16, 3 plain text floats).

3.6.5.4 convertalis

Usage

`foldseek convertalis <i:queryDb> <i:targetDb> <i:alignmentDB> <o:alignmentFile> [options]`

`convertalis` is the primary result exporter. It converts internal alignment DBs to tabular, SAM, HTML, or superposed-structure style outputs and can expose structural metrics such as TM-score and LDDT columns.

`--format-output` is central for custom analysis pipelines. When requesting columns like `alntmscore`, `qtmscore`, `ttmscore`, `lddt`, `u`, or `t`, `convertalis` triggers the required internal computations from stored coordinates and backtraces.

3.6.5.5 convert2pdb

Usage

```
foldseek convert2pdb <i:Db> <o:pdbFile|pdbDir> [options]
```

convert2pdb materializes superposed C-alpha coordinates into PDB output. --pdb-output-mode selects whether output is one multi-model file, per-chain files, or per-complex files.

3.6.5.6 aln2tmscore

Usage

```
foldseek aln2tmscore <i:queryDB> <i:targetDB> <i:alnDB> <o:resultDB> [options]
```

aln2tmscore converts existing alignment DBs into TM-score-focused result DBs. This is useful when TM-score conversion is needed as a standalone post-processing stage.

3.6.5.7 result2profile

Usage

```
foldseek result2profile <i:queryDB> <i:targetDB> <i:resultDB> <o:profileDB> [options]
```

result2profile builds profile DBs from result alignments. It is mainly relevant for iterative/profile-oriented workflows where profile diversity and redundancy controls (--max-seq-id, --qid, --cov, --diff) matter.

3.6.5.8 Typical Data-Preparation Flow

```
foldseek createdb structures/ structDB
foldseek createindex structDB tmp
foldseek search structDB targetDB alnDB tmp
foldseek convertalis structDB targetDB alnDB result.tsv --format-output
query,target,alntmscore,lddt,bits,evalue
```

This flow keeps ingestion, indexing, search, and conversion explicitly separated, which makes reruns and parameter sweeps easier to reproduce.

3.6.6 Database Management Modules

Foldseek performance is tightly coupled to how databases are stored on disk. The search and clustering workflows do not read one monolithic file; they read coordinated channels (DB, DB_ss, DB_ca, DB_h) plus index and metadata sidecars. This chapter documents those files as an operational format specification, then explains how databases, createindex, and createclusearchdb manipulate that layout.

3.6.6.1 Canonical Database Layout

A standard createdb output with prefix X produces a synchronized family of files:

Path	Role	Primary readers
X, X.index, X.dbtype	Amino-acid channel (one entry per chain)	search, structurealign, cluster, convertalis
X_ss, X_ss.index, X_ss.dbtype	3Di channel, residue-aligned to X	prefilter/alignment stages

Path	Role	Primary readers
X_ca, X_ca.dbtype	X_ca.index, C-alpha coordinates for TM/LDDT/structure-bit sorting	structurealign, tmalalign, converters
X_h, X_h.index, X_h.dbtype	Header channel (entry identifiers/metadata text)	converters/reporting
X.lookup	Entry mapping id -> accession -> fileNumber	lookup-based conversion and tooling
X.source	fileNumber -> source file/model label	provenance-aware exports
X_mapping (optional)	id -> taxId mapping (-- write-mapping 1)	taxonomy-aware downstream workflows

The key design invariant is shared IDs across channels: for a given internal key, X, X_ss, X_ca, and X_h describe the same chain entry.

3.6.6.2 Core File Formats

<DB>.index is a tab-separated text index. Each line is:

```
<id>\t<offset>\t<length>\n
```

id is the internal key, offset points into <DB> data payload, and length is the stored entry length. This format is written in DBWriter::indexToBuffer (MMseqs2/src/commons/DBWriter.cpp) and parsed by DBReader::readIndex (MMseqs2/src/commons/DBReader.cpp).

<DB>.dbtype is a 4-byte integer (int32) describing logical DB type plus compression flag. The high bit (bit 31) indicates compressed storage; the lower bits hold the base DB type (DBTYPE_AMINO_ACIDS, DBTYPE_GENERIC_DB, Foldseek DBTYPE_CA_ALPHA=101, and so on).

<DB> stores concatenated entry payloads separated by a null terminator. In compressed writer mode, per-entry framing is handled by the writer and the same index file still resolves entry boundaries.

<DB>.lookup is line-based text with exactly three columns:

```
<id>\t<entryName>\t<fileNumber>\n
```

This is the exact serializer used by DBReader<unsigned int>::lookupEntryToBuffer in MMseqs2/src/commons/DBReader.cpp.

<DB>.source maps fileNumber to an original source label:

```
<fileNumber>\t<sourceName>\n
```

It is written during createdb lookup generation in foldseek/src/strucclustutils/structcreatedb.cpp.

3.6.6.3 Channel Payload Semantics

For createdb outputs, the payload format inside each channel is:

Channel	Entry payload
X	Amino-acid one-letter sequence plus trailing newline (\n)
X_ss	3Di token sequence (one residue, one symbol) plus trailing newline
X_h	Header text plus trailing newline
X_ca	Binary or text coordinate encoding (--coord-store-mode)

X and X_ss are residue-synchronized: position i in X_ss describes the same residue as position i in X. That invariant is what enables 3Di+AA alignment (--alignment-type 2) without conversion steps.

3.6.6.4 C-alpha Storage Modes (X_ca)

X_ca has three encodings, selected at creation time (createdb) or rewritten later (compressca).

Mode	Option	On-disk payload	Notes
Float	--coord-store-mode 1	3 * L float32 values (x[0..L-1], y[0..L-1], z[0..L-1])	Highest compatibility, largest footprint
Diff16	--coord-store-mode 2	Per axis: start:int32 + (L-1) deltas int16; packed for x, y, z, plus one trailing compatibility byte	Default mode in Foldseek; values are scaled by 1000 before integer encoding
Plain text	compressca --coord-store-mode 3	Comma-separated float list + newline	Stored as generic DB type; useful for inspection/interchange

In Diff16 mode, createdb attempts integer delta encoding and falls back to float payload for entries that overflow int16 during conversion. Decoding is handled by `Coordinate16::read (foldseek/src/commons/Coordinate16.h)`.

3.6.6.5 databases

Usage

`foldseek databases <name> <o:sequenceDB> <tmpDir> [options]`

databases downloads curated structural datasets and model assets (AlphaFold subsets, PDB snapshots, BFMD/BFVD, ProstT5 weights). Use `--tsv 1` when you need a machine-readable catalog for reproducible provisioning scripts.

3.6.6.6 createindex

Usage

```
foldseek createindex <i:sequenceDB> <tmpDir> [options]
```

`createindex` is a Foldseek workflow wrapper around MMseqs indexing plus Foldseek-specific channel handling (`foldseek/src/workflow/StructureIndex.cpp` and `foldseek/data/structureindex.sh`). Operationally it does three things:

1. Builds index data for the AA channel and the 3Di channel.
2. Links header/cluster side channels needed by Foldseek workflows.
3. Optionally appends C-alpha channels into the `.idx` container unless excluded.

The two high-impact controls are:

Option	Effect
<code>--index-subset</code>	Drops selected index components (1: no headers, 2: no prefilter data, 4: no alignment payloads) to reduce index size.
<code>--index-exclude</code>	Bit flags for Foldseek-specific exclusions (1: omit k-mer index, 2: omit C-alpha append into index).

When `--index-exclude 2` is used, coordinate-aware ranking and threshold paths may be disabled later unless raw `_ca` side databases are still available.

3.6.6.7 createclusearchdb

Usage

```
foldseek createclusearchdb <i:sequenceDB> <i:clusterDB> <o:sequenceDB> [options]
```

`createclusearchdb` transforms clustering output into representative/member search layout for `--cluster-search 1`. For each selected suffix (default `_h`, `_ss`, `_ca` in Foldseek help), it writes:

- representative channel: `<out><suffix>`
- member channel: `<out>_seq<suffix>`

It also copies cluster topology to `<out>_clu` and propagates metadata sidecars (`.lookup`, `.source`, `_mapping`, taxonomy files) with aliases for `_seq` views.

A source-level detail that matters for debugging: `_seq` channels are assembled as a two-part DB where representatives and non-representatives can live in separate backing files, while one merged `_seq.index` maps IDs to the correct offsets. This is normal and expected.

For cluster-search compatibility, `StructureSearch.cpp` checks for at least:

- `<target>_seq`, `<target>_seq_ss`, `<target>_seq_h`
- one of `<target>_clu` or `<target>_aln`

If coordinate-aware ranking/filtering is required, keep `<target>_seq_ca` available as well.

3.6.6.8 Practical Layout Strategies

If you run repeated searches on a fixed target, build and validate indices once (`--check-compatible 1`) and keep full coordinate channels. If you are strictly ungapped or no-structure-bit ranking, excluding selected index parts can reduce RAM and startup time, but apply those exclusions only after confirming that downstream modules do not need TM/LDDT or structure-bit sorting.

3.6.6.9 Minimal End-to-End Example

```
foldseek databases Alphafold/Swiss-Prot afdb tmp
foldseek createindex afdb tmp --check-compatible 1
foldseek search queryDB afdb alnDB tmp
```

For sequence-to-3Di prediction workflows:

```
foldseek databases ProstT5 weights tmp
foldseek createdb query.fasta queryDB --prostt5-model weights
```

4 Foldseek Expert Manual

This chapter explains the algorithmic and systems design decisions that determine Foldseek behavior at scale. Command syntax and user-facing workflows are covered in the manual and module chapters; here the focus is model internals, scoring semantics, and data-path constraints.

4.1 3Di Encoding Engine

Foldseek converts structure geometry to discrete 3Di symbols in `foldseek/lib/3di/structureto3di.cpp`. The implementation is a staged encoder rather than a direct heuristic labeler: virtual centers are computed from backbone/side-chain atoms, residue partners are selected, ten geometric features are assembled per pair, features are projected into a learned low-dimensional embedding, and the embedding is quantized to one of twenty centroids.

That architecture is central to throughput. The prefilter does not operate on raw coordinates, and it does not need dynamic structural superposition during candidate generation. It operates on a residue-aligned symbolic stream that preserves enough local geometry to remain discriminative while keeping indexing and k-mer operations cheap.

4.2 Prefilter and Candidate Generation

For large databases, the prefilter is still the main runtime gate. Foldseek inherits MMseqs-style index and k-mer mechanics, but applies them to 3Di channels (`_ss`) and Foldseek-specific workflow defaults.

Two options define most candidate behavior:

- `--target-search-mode` chooses exact k-mer search or similar-k-mer expansion.

- `--exact-kmer-matching 1` disables similar-k-mer expansion and forces strict seed matching.
- `--prefilter-mode` then controls how candidates are refined before full alignment. In workflow code (`foldseek/src/workflow/StructureSearch.cpp`), GPU mode can enforce ungapped-first behavior when other prefilter combinations are incompatible. This is one reason CPU and GPU runs can produce different candidate sets under nominally similar settings.

4.3 Alignment Core: 3Di, 3Di+AA, and TM-align

Foldseek supports three alignment families through `--alignment-type`.

Type 0 uses 3Di local alignment; type 1 uses TM-align-style structural superposition; type 2 combines 3Di and amino-acid evidence in one local alignment objective. In `foldseek/src/strucclustutils/structurealign.cpp`, type 0 and 2 run through `StructureSmithWaterman` (`foldseek/src/commons/StructureSmithWaterman.cpp`) with vectorized query profiles and forward/reverse scoring.

The combined 3Di+AA path is not a cosmetic rerank. It changes the score landscape during alignment itself, so candidate acceptance, tie-breaking, and downstream ranking can differ substantially from pure 3Di or pure TM-oriented runs.

4.4 TM-score and LDDT in Filtering and Ranking

TM-score and LDDT are integrated into runtime filtering and sorting, not only reporting. `TMaligner` (`foldseek/src/commons/TMaligner.cpp`) and `LDDTCalculator` (`foldseek/src/commons/LDDT.cpp`) are invoked in alignment workflows when thresholds or structure-aware sorting require coordinate-derived quality terms.

When `--sort-by-structure-bits 1` is active, `structurealign.cpp` applies a composite structural ranking term ($\text{score} * \sqrt{\text{lddt} * \text{tmscore}}$). If required C-alpha channels are missing, Foldseek disables incompatible thresholds/sorting paths with warnings instead of silently producing invalid rankings.

4.5 Multimer Assignment and Complex Scoring

Foldseek-Multimer builds complex scores from chain-level evidence in explicit stages.

`expandmultimer` generates expanded chain-pair candidates. `scoremultimer` then groups those hits by query/target complex and solves assignment consistency before calculating complex-level scores. In `scoremultimer.cpp`, assignment selection uses clustering and chain reuse controls so one local chain optimum does not destabilize the whole complex mapping.

Multimer filtering is implemented inside `scoremultimer.cpp` (`ComplexFilter`), not in a separate module. The filter stack combines global complex TM, coverage, per-chain TM, and interface LDDT gates. This is why thresholds such as `--multimer-tm-threshold`, `--chain-tm-threshold`, and `--interface-lddt-threshold` are complementary rather than interchangeable. For command-level details and field semantics, see Multimer Modules.

4.6 Data Layout, Storage Format, and Memory Behavior

Foldseek databases are multi-channel by design. A single logical entry is distributed across AA (DB), 3Di (DB_ss), coordinates (DB_ca), and headers (DB_h), all keyed by the same internal ID. This lets workflows load only the channels needed by the current stage while preserving residue alignment between AA and 3Di streams.

4.6.1 On-disk Containers and Sidecars

Every channel uses the same MMseqs container pattern: <name>, <name>.index, <name>.dbtype. The index stores <id, offset, length> triplets; .dbtype stores type and compression flags; optional .lookup and .source sidecars encode identifier and provenance mappings.

For format-level details (line and payload layout), see Database Management: Core File Formats.

4.6.2 C-alpha Encoding Trade-offs

DB_ca can be stored as float32 coordinates or as delta-encoded int16 (--coord-store-mode 2, default). Delta mode stores one int32 start value plus int16 increments per residue and per axis, which cuts footprint and I/O while preserving enough precision for downstream scoring. If a chain overflows int16 deltas during creation, Foldseek falls back to float storage for that entry.

A practical implication is mixed storage at entry granularity: one database can contain mostly diff16 entries plus float fallbacks. The decoder (Coordinate16::read) handles both by inspecting entry length.

4.6.3 Indexed Targets and Exclusion Flags

createindex on Foldseek databases is not a single index build; it is a workflow that indexes AA/3Di channels and can append coordinate channels into the .idx container. Exclusion flags (--index-exclude) and subset flags (--index-subset) reduce footprint but remove capabilities.

The important compatibility rule is simple: if ranking or filtering needs coordinate-derived metrics (TM/LDDT/structure-bit sorting), coordinate channels must remain reachable either as side DBs (_ca) or appended index components.

4.6.4 Cluster-search Data Model

createclusearchdb materializes representative/member split channels (<db> and <db>_seq*) plus cluster topology (<db>_clu). Cluster-search workflows (--cluster-search 1) depend on this layout to search representatives first and expand to members without rebuilding mappings at query time.

That split is a major speed lever for very large clustered targets: representative pruning reduces expensive downstream alignment volume, while member expansion restores sensitivity in the final result set.

4.7 GPU Paths and Constraints

Foldseek exposes two major GPU paths:

1. Search workflow acceleration with --gpu where compatible prefilter/alignment combinations exist.

2. ProstT5-backed 3Di prediction during `createdb` when `--prostt5-model` is provided and GPU backends are available.

Workflow code enforces compatibility constraints (especially around prefilter mode). For reproducible benchmarking, always report CPU/GPU mode, prefilter mode, and alignment type together.

4.8 Practical Tuning Order

For production optimization, tune in dependency order instead of isolated flags.

1. Control candidate volume first (`-s`, `--max-seqs`, `--prefilter-mode`, k-mer mode).
2. Choose alignment objective (`--alignment-type`) and structural thresholds.
3. Choose ranking semantics (`--sort-by-structure-bits`, TM/LDDT columns).
4. Only then trim storage/index footprint (`--index-subset`, `--index-exclude`, coordinate mode).

Most wall-clock gains come from reducing candidate volume and avoiding incompatible scoring paths early, not from late-stage filtering changes alone.

5 Foldseek Developer Manual

This chapter documents Foldseek source architecture and extension points. It is for contributors modifying Foldseek behavior, not for user-level command tutorials.

5.1 Command Registration and Ownership

Public command registration is centralized in `foldseek/src/FoldseekBase.cpp`. This file is the first source of truth for:

- command visibility (main/easy/hidden categories),
- usage signatures,
- command descriptions and examples,
- command-to-function entrypoint mapping.

When behavior changes but docs are stale, check `FoldseekBase.cpp` first, then the target implementation.

5.2 Parameter System

Foldseek-specific parameters and workflow parameter bundles are defined in `foldseek/src/commons/LocalParameters.cpp` and declared in `LocalParameters.h`.

This layer controls:

- option defaults and validation regexes,
- semantic groupings (prefilter, align, clust, misc, expert),
- workflow parameter composition (`structuresearchworkflow`, `multimersearchworkflow`, easy workflow bundles),
- output-column dependency flags used by converters.

If an option appears in help but not in expected workflow behavior, inspect the workflow's parameter bundle composition in `LocalParameters.cpp`.

5.3 Workflow Layer

Workflow commands live in `foldseek/src/workflow/` and primarily orchestrate stage composition rather than numerical kernels.

File	Responsibility
<code>StructureSearch.cpp</code>	Main search workflow composition and mode switching.
<code>StructureCluster.cpp</code>	Structural clustering workflow orchestration.
<code>StructureRbh.cpp</code>	Reciprocal-best-hit workflow.
<code>MultimerSearch.cpp</code>	Complex-level search orchestration.
<code>MultimerCluster.cpp</code>	Complex clustering orchestration.
<code>Easy*.cpp</code>	Raw-input wrappers that call <code>createdb</code> + workflow stages + conversion.
<code>StructureIndex.cpp</code>	Index workflow wrapper for Foldseek DBs.

Most “why did this pipeline run this module” questions are answered in these files and their generated shell templates under `foldseek/data/`.

5.4 Algorithm and Utility Layer

Structural kernels and transformations are in `foldseek/src/strucclustutils/`.

Area	Key files
Structure DB creation	<code>structcreatedb.cpp</code> , <code>structureto3didescriptor.cpp</code> , <code>GemmiWrapper.*</code> , <code>PulchraWrapper.*</code>
Search alignment kernels	<code>structurealign.cpp</code> , <code>structurerescorediagonal.cpp</code> , <code>talign.cpp</code>
Conversion/reporting	<code>structureconvertalis.cpp</code> , <code>convert2pdb.cpp</code> , <code>createmultimerreport.cpp</code>
Multimer internals	<code>expandmultimer.cpp</code> , <code>scoremultimer.cpp</code> , <code>MultimerUtil.h</code>
Utility transforms	<code>compressca.cpp</code> , <code>aln2tmscore.cpp</code> , <code>result2structprofile.cpp</code>

When changing scoring semantics, always check both runtime modules and conversion modules; output columns may recompute or reinterpret metrics.

5.5 Core Libraries and Shared Engines

Foldseek reuses MMseqs infrastructure but adds structural engines in dedicated components:

Component	Files
3Di encoder	foldseek/lib/3di/structureto3di.*
TM-align backend	foldseek/lib/tmalign/*, foldseek/src/commons/TMaligner.*
LDDT scoring	foldseek/src/commons/LDDT.*
Structure-aware SW	foldseek/src/commons/StructureSmithWaterman.*
Shared command/DB infra	MMseqs-derived DBReader, DBWriter, IndexReader, Parameters

The 3Di encoder is isolated from workflow code, which allows algorithm updates without rewriting orchestration paths.

5.6 End-to-End Data Flow (Developer View)

1. createdb builds synchronized DB channels from structures or ProST5 inference output.
2. Workflow module (search, cluster, multimersearch, etc.) composes prefilter/alignment stages.
3. Alignment/result DBs are written in internal formats.
4. Conversion/report modules derive user-facing outputs.

Because multiple commands share the same DB channels, schema changes in createdb impact almost every downstream module.

5.7 Editing Guidelines

1. For command behavior changes, update all three layers together:
 - FoldseekBase.cpp command declaration,
 - LocalParameters.cpp option bundles/defaults,
 - implementation file(s).
2. For score or ranking changes, validate:
 - runtime filtering paths (structurealign, structurerescorediagonal, multimer filters),
 - conversion paths (structureconvertalis, reports),
 - sorting/threshold side effects.
3. For DB/index changes, test both:
 - non-indexed and indexed targets,
 - coordinate-dependent and coordinate-light modes (--sort-by-structure-bits, TM/LDDT outputs).

5.8 Debugging Map

Symptom	First files to inspect
Wrong command usage/help	foldseek/src/FoldseekBase.cpp, regenerated foldseek_help_output/*.txt
Option accepted but ignored	foldseek/ src/commons/LocalParameters.cpp, workflow bundle composition
Unexpected stage order	foldseek/src/workflow/*.cpp, generated workflow shell templates
Score/ranking mismatch	structurealign.cpp, structurerescorediagonal.cpp, StructureSmithWaterman.*, TAligner.*, LDDT.*
Multimer assignment anomalies	scoremultimer.cpp, expandmultimer.cpp, MultimerUtil.h
Output field inconsistency	structureconvertalis.cpp, createmultimerreport.cpp

5.9 Documentation Sync Rule

Whenever command interfaces or defaults change, refresh help snapshots with `generate_foldseek_docs.sh` and reconcile the module chapters before generating `foldseek_doc.pdf`.